Apuntes de JavaScript

2DAW — Desarrollo en Entorno Cliente

Basado en javascript.info (es.javascript.info)

2.1 · ¡Hola, mundo!

JavaScript se ejecuta en el navegador (u otros entornos, como Node.js). Para incluir código JS en una página HTML se usa la etiqueta <script>.

La etiqueta <script>

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Antes del script...</p>
  <script>
    alert('¡Hola, mundo!');
  </script>
  <p>...después del script.</p>
</body>
</html>
```

El código dentro de <script> se ejecuta automáticamente en cuanto el navegador llega a esa línea. No hace falta poner type="text/javascript" ni language="...": son restos de HTML antiguo, hoy innecesarios.

Scripts externos

Si el código es largo, conviene guardarlo en un archivo .js aparte y enlazarlo con el atributo src:

```
<script src="/js/script.js"></script>
```

- Ventaja: el navegador puede guardar el archivo en caché, así que páginas distintas que usan el mismo script no lo vuelven a descargar.
- Un mismo <script> no puede tener src y código dentro a la vez: si hay src, el contenido interior se ignora.

2.2 · Estructura del código

Sentencias

Una sentencia es una instrucción que realiza una acción (por ejemplo, alert('Hola')). Se pueden separar con punto y coma, aunque lo habitual es escribir una sentencia por línea:

```
alert('Hola');
alert('Mundo');
```

Punto y coma: ¿obligatorio?

JavaScript suele insertar el punto y coma automáticamente al final de una línea (ASI, Automatic Semicolon Insertion), por lo que muchas veces funciona sin escribirlo. Pero esta detección automática no es perfecta y en ciertos casos provoca errores difíciles de encontrar (por ejemplo, cuando una línea empieza con corchetes [...]).

> 💡 *Por seguridad, sobre todo si estás empezando, escribe siempre el punto y coma al final de cada sentencia.*

Comentarios

- Comentario de una línea: empieza con // y todo lo que sigue en esa línea se ignora.
- Comentario de varias líneas: se escribe entre /* y */.

```
// Esto es un comentario de una línea
alert('Hola');

/* Esto es un
   comentario de varias líneas */
alert('Mundo');
```

Los comentarios no se ejecutan y no afectan al resultado del programa: son solo documentación para quien lee el código (¡y para tu propio yo del futuro!). No se pueden anidar comentarios de bloque dentro de otros comentarios de bloque.

2.3 · El modo moderno, "use strict"

En 2009 apareció ECMAScript 5 (ES5), que introdujo mejoras en el lenguaje. Para no romper páginas antiguas, esas mejoras solo se activan si se indica explícitamente con la directiva "use strict".

```
"use strict";

// a partir de aquí el código funciona en modo moderno
```

> 💡 *"use strict" debe ir en la primera línea real del script (solo pueden ir comentarios por encima). Si se coloca más abajo, no tiene ningún efecto.*

No existe una forma de "desactivar" el modo estricto una vez activado. Además, si el código moderno usa clases o módulos, el modo estricto se activa automáticamente y no hace falta escribir la directiva.

> 💡 *Recomendación práctica: mientras no trabajes con clases ni módulos, empieza siempre tus scripts con "use strict";. Es el comportamiento que se asume en el resto de estos apuntes.*

2.4 · Variables

Una variable es un "contenedor con nombre" donde guardamos datos. En JavaScript moderno se declaran con let (o const para valores que no cambian).

```
let message;
message = 'Hola';

// o en una sola línea:
let message2 = 'Hola';
```

```
alert(message2); // Hola
```

También se pueden declarar varias variables juntas, pero se recomienda una por línea para que el código sea más legible:

```
let nombre = 'Ana';
let edad = 20;
let ciudad = 'Madrid';
```

Reglas para nombrar variables

- Solo pueden contener letras, dígitos, y los símbolos $ y _.
- El primer carácter no puede ser un dígito.
- Se distingue entre mayúsculas y minúsculas: user y USER son variables distintas.
- Se suele usar el estilo camelCase: miVariableDeEjemplo.
- No se pueden usar palabras reservadas del lenguaje (let, class, return, function...).

> 💡 *Una variable solo puede declararse una vez con let en el mismo bloque. Volver a declararla con let da un SyntaxError.*

Constantes (const)

Si un valor no debe cambiar nunca, se declara con const en lugar de let. Intentar reasignarla produce un error:

```
const miCumpleanos = '18.04.2005';
miCumpleanos = '01.01.2001'; // ¡Error! No se puede reasignar una constante
```

Por convención, las constantes cuyo valor se conoce de antemano (antes de ejecutar el programa) se escriben en MAYÚSCULAS_CON_GUIONES_BAJOS, por ejemplo COLOR_ROJO. Las constantes calculadas durante la ejecución (aunque no cambien después) se nombran en minúsculas normales.

Buenas prácticas al nombrar

- Usa nombres claros y legibles: userName, shoppingCart.
- Evita nombres demasiado cortos o poco descriptivos como a, b, data, value.
- Sé consistente: si a un visitante lo llamas 'user', usa currentUser, no currentVisitor.

6.3 · Ámbito de variables: "Bloques de código"

Cuando una variable se declara con let o const dentro de un bloque {...}, solo existe (es visible) dentro de ese bloque. Fuera de él, esa variable no existe.

```
{
  let message = 'Hola';
  alert(message); // Hola
}

alert(message); // Error: message no está definido
```

Esto permite reutilizar el mismo nombre de variable en bloques distintos sin que choquen entre sí:

```
{
  let message = 'Hola';
  alert(message);
}

{
  let message = 'Adiós'; // otra variable, mismo nombre, otro bloque
  alert(message);
}
```

Esta misma regla se aplica a los bloques de if, for y while: una variable declarada con let dentro de sus llaves solo vive ahí dentro.

```
if (true) {
  let frase = '¡Hola!';
  alert(frase); // funciona, frase existe aquí
}

alert(frase); // Error, frase no existe fuera del bloque if
```

```
for (let i = 0; i < 3; i++) {
  // i solo es visible dentro de este bucle for
  alert(i); // 0, 1, 2
}

alert(i); // Error, i no existe aquí fuera
```

💡 *Piensa en cada { ... } como una 'sala' independiente: las variables declaradas con let dentro de una sala no se pueden ver desde fuera de ella.*

6.4 · La vieja "var"

var es la forma antigua de declarar variables, anterior a let y const. Hoy en día no se usa en código nuevo, pero es importante conocerla porque aparece en scripts antiguos. Su comportamiento es diferente al de let en varios aspectos clave.

1. var no respeta el ámbito de bloque

A diferencia de let, una variable declarada con var dentro de un bloque {...} (if, for, etc.) sigue siendo visible fuera de ese bloque:

```
if (true) {
  var test = true;
}

alert(test); // true → var 'se escapa' del bloque if
```

var solo tiene dos posibles ámbitos: el de la función donde se declara, o el ámbito global si está fuera de cualquier función. Los bloques {...} no la detienen.

2. var permite redeclaraciones

Mientras que let user; let user; da error, con var se puede declarar la misma variable varias veces sin que salte ningún error (la segunda declaración simplemente se ignora):

```
var user = 'Pedro';
var user = 'Juan'; // no da error, simplemente no hace nada nuevo
alert(user); // Juan
```

3. "Hoisting": var se procesa antes de ejecutarse

Las declaraciones (que no las asignaciones) de var se "elevan" mentalmente al principio de la función o del script, aunque estén escritas más abajo en el código:

```
function saludar() {
  alert(frase); // undefined (no da error, pero tampoco tiene valor aún)
  var frase = 'Hola';
}
saludar();
```

Es como si el motor de JavaScript reorganizara el código así por dentro: primero coloca var frase; al principio de la función, y deja la asignación frase = 'Hola'; en su sitio original.

> 💡 *Resumen rápido: usa siempre let y const en código nuevo. var puede provocar errores sutiles (variables que 'se escapan' de los bloques, redeclaraciones silenciosas) que let evita por diseño.*

2.5 · Tipos de datos

JavaScript es un lenguaje de tipado dinámico: una variable puede contener un valor de cualquier tipo, y ese tipo puede cambiar durante la ejecución. Hay 8 tipos de datos básicos.

Tipo	Qué representa
number	Números enteros y decimales. Incluye valores especiales: Infinity, -Infinity y NaN (resultado de un cálculo inválido).
bigint	Enteros de tamaño arbitrariamente grande, mayores de lo que number puede representar con precisión. Se escriben añadiendo n al final: 123n.
string	Cadenas de texto, entre comillas dobles, simples o backticks (`). Solo los backticks permiten incrustar variables con ${...}.
boolean	Solo dos valores posibles: true o false.
null	Valor especial que indica 'vacío' o 'desconocido a propósito'. Es su propio tipo.
undefined	Valor que toma una variable declarada pero aún no asignada. Significa 'valor no asignado'.
object	Estructuras de datos más complejas, colecciones de valores relacionados.
symbol	Identificadores únicos, se usa para casos avanzados.

El operador typeof

Sirve para comprobar de qué tipo es un valor:

```
typeof 123        // "number"
typeof 'hola'      // "string"
typeof true        // "boolean"
typeof undefined   // "undefined"
typeof null        // "object" (esto es un error histórico del lenguaje;
                   //  null NO es realmente un objeto)
```

2.6 · Interacción: alert, prompt, confirm

Son tres funciones del navegador (no del lenguaje JavaScript en sí) para interactuar con quien visita la página. Las tres son modales: detienen la ejecución del script hasta que el usuario responde.

Función	Qué hace	Devuelve
`alert(msg)`	Muestra un mensaje y espera a que se pulse "Aceptar".	undefined
`prompt(txt, def)`	Pide un texto al usuario, con un valor inicial opcional (def).	El texto escrito, o null si se cancela
`confirm(pregunta)`	Muestra una pregunta con botones OK/Cancelar.	true (OK) o false (Cancelar)

```
let edad = prompt('¿Cuántos años tienes?', 18);
alert(`Tienes ${edad} años`);

let esJefe = confirm('¿Eres el jefe?');
alert(esJefe); // true si pulsó OK, false si pulsó Cancelar
```

> 💡 *El aspecto visual de estas ventanas (posición, colores...) lo decide el navegador, no se puede personalizar con CSS.*

2.7 · Conversiones de tipos

Muchas veces JavaScript convierte los valores de tipo automáticamente, pero también podemos forzar una conversión explícita con String(), Number() y Boolean().

A texto: String(valor)

La conversión más sencilla: cualquier valor se convierte en su representación textual.

```
let valor = true;
let texto = String(valor);
alert(texto); // "true"
```

A número: Number(valor)

Se usa, por ejemplo, cuando leemos un dato de un formulario (que llega como texto) y necesitamos tratarlo como número.

Valor original	Se convierte en
`undefined`	NaN
`null`	0
`true / false`	1 / 0
`"123"`	123 (se quitan espacios sueltos al inicio/fin)
`"123abc"`	NaN (texto no numérico válido → error de conversión)

A booleano: Boolean(valor)

- Se convierten en false (valores "falsy"): 0, "" (texto vacío), null, undefined, NaN.
- Cualquier otro valor se convierte en true, incluido el string "0" (¡ojo! en JS, a diferencia de otros lenguajes, "0" es truthy).

```
Boolean(1)     // true
Boolean(0)     // false
Boolean('')    // false
Boolean('0')   // true (¡es un string no vacío!)
Boolean(' ')   // true (solo espacios, pero no está vacío)
```

2.8 · Operadores básicos, matemáticas

Operaciones matemáticas

- Suma +, resta -, multiplicación *, división /
- Resto de la división %: 5 % 2 da 1 (lo que sobra al dividir 5 entre 2).
- Exponenciación **: 2 ** 3 da 8 (2 elevado a 3).

El + con cadenas de texto

Si se usa + con al menos un string, el otro operando se convierte también a string y se concatenan (se unen), en vez de sumarse:

```
alert('1' + 2);      // "12"
alert(2 + 2 + '1');  // "41" (primero 2+2=4, luego se concatena '1')
alert('1' + 2 + 2);  // "122" (de izquierda a derecha, todo se concatena)
```

> 💡 *El resto de operadores matemáticos (-, *, /) sí convierten siempre sus operandos a número, aunque sean strings: '6' / '2' da 3.*

Incremento y decremento

- counter++ (sufijo): devuelve el valor ANTES de incrementar, y luego incrementa.
- ++counter (prefijo): incrementa primero, y devuelve el valor YA incrementado.

```
let a = 1;
let b = a++; // b = 1 (valor antiguo), luego a pasa a valer 2
```

```
let c = 1;
let d = ++c; // c pasa a valer 2 primero, d = 2 (valor nuevo)
```

Operadores "modifica y asigna"

Forma abreviada de aplicar una operación a una variable y guardar el resultado en ella misma:

```
let n = 2;
n += 5; // igual que n = n + 5 → n vale 7
n *= 2; // igual que n = n * 2 → n vale 14
```

2.9 · Comparaciones

Los operadores de comparación (>, <, >=, <=, ==, !=) siempre devuelven un valor booleano (true o false).

```
alert(2 > 1);  // true
alert(2 == 1); // false
```

Comparación de textos

Las cadenas se comparan letra por letra, siguiendo el orden Unicode (parecido al orden de un diccionario, pero no idéntico: las mayúsculas y minúsculas tienen distinto valor).

== vs ===: igualdad débil y estricta

- == (igualdad simple): si los tipos son distintos, convierte los valores a número antes de comparar.
- === (igualdad estricta): compara también el tipo; si los tipos son distintos, siempre da false, sin convertir nada.

```
alert(0 == false);   // true  → false se convierte en 0
alert(0 === false);  // false → distinto tipo, no se convierte
alert('' == false);  // true
alert('' === false); // false
```

> 💡 *Recomendación: usa === (y !==) en la mayoría de los casos. Es más predecible y evita sorpresas por conversiones automáticas de tipo.*

Casos especiales: null y undefined

- null == undefined da true (se consideran 'iguales' entre sí, y solo entre sí).
- null === undefined da false (distinto tipo).
- En comparaciones como >, <, >=, <=, null se convierte en 0 y undefined en NaN (que nunca es mayor, menor ni igual a nada).

```
alert(null > 0);   // false
alert(null == 0);  // false
alert(null >= 0);  // true  ← puede resultar sorprendente
```

> 💡 *Por esta rareza, evita comparar null o undefined con > < >= <= salvo que sepas exactamente lo que haces. Mejor comprobarlos aparte.*

2.10 · Ejecución condicional: if, '?'

La sentencia "if"

Ejecuta un bloque de código solo si la condición entre paréntesis es verdadera (o se convierte a truthy):

```
let edad = 20;

if (edad >= 18) {
  alert('Eres mayor de edad');
}
```

> 💡 *Se recomienda usar siempre llaves {} con if, aunque el bloque tenga una sola sentencia: mejora la legibilidad y evita errores al añadir código luego.*

"else" y "else if"

```
let nota = 6;

if (nota >= 9) {
  alert('Sobresaliente');
} else if (nota >= 7) {
  alert('Notable');
} else if (nota >= 5) {
  alert('Aprobado');
} else {
  alert('Suspenso');
}
```

Operador ternario ?

Permite escribir de forma compacta un if/else que devuelve un valor. Su sintaxis es: condición ? valorSiVerdad : valorSiFalso

```
let edad = 20;
let acceso = (edad >= 18) ? 'permitido' : 'denegado';
alert(acceso); // permitido
```

> 💡 *El operador ? está pensado para elegir entre dos VALORES, no para sustituir un if normal que ejecuta acciones. Usarlo así (en vez de if) funciona, pero hace el código menos legible; se recomienda evitarlo.*

2.11 · Operadores lógicos

JavaScript tiene tres operadores lógicos principales: || (OR / O), && (AND / Y) y ! (NOT / NO). Aunque se llaman 'lógicos', pueden trabajar con valores de cualquier tipo, no solo booleanos.

|| (OR): devuelve el primer valor truthy

No se limita a dar true/false: evalúa de izquierda a derecha y devuelve el primer valor 'verdadero' (truthy) que encuentra. Si todos son falsy, devuelve el último.

```
alert(1 || 0);              // 1 (primer valor truthy)
alert(null || 0 || 'hola'); // 'hola' (primer valor truthy)
alert(null || undefined);   // undefined (todos falsy → se devuelve el último)
```

Uso típico: elegir el primer valor 'con contenido' entre varias variables opcionales:

```
let apodo = '';
let nombre = 'Ana';
alert(apodo || nombre || 'Anónimo'); // 'Ana'
```

&& (AND): devuelve el primer valor falsy

Al contrario que ||: evalúa de izquierda a derecha y devuelve el primer valor 'falso' (falsy) que encuentra. Si todos son truthy, devuelve el último.

```
alert(1 && 5);        // 5 (todos truthy → devuelve el último)
alert(1 && 0 && 3);   // 0 (primer valor falsy)
alert(null && 5);     // null (primer valor falsy, y el único)
```

> 💡 *&& tiene más precedencia que ||, así que en una expresión como a && b || c && d, se evalúan antes las partes con &&.*

! (NOT): invierte el valor booleano

Convierte el valor a booleano y devuelve el contrario. Usarlo dos veces seguidas (!!valor) es un truco muy común para convertir cualquier valor a su equivalente booleano:

```
alert(!true);   // false
alert(!0);      // true
alert(!!'texto'); // true  (equivale a Boolean('texto'))
alert(!!null);    // false (equivale a Boolean(null))
```

> 💡 *No conviene usar && u || como si fueran un 'if' abreviado (por ejemplo: (x > 0) && alert('positivo')). Aunque funciona, un if normal es más claro y menos propenso a errores.*

Fin de los apuntes · Fuente: es.javascript.info (secciones 2.1–2.11 y 6.3–6.4)